Herald

Herald Constitution

Field	Value
Revision	13
Created	2026-05-15
Last modified	2026-05-31
Status	active
Status summary	r13: extended §108 with §108.p restating inherited HelixConstitution §11.4.106 (Docs Chain documentation-sync mandate, 2026-05-29) at Herald project-constitution level per §1.1 multi-file propagation discipline; inherited per §11.4.35, restated + cited, not redefined; literal anchor `11.4.106` now present in all four Herald governance docs (CLAUDE.md r19, AGENTS.md r16, QWEN.md, HERALD_CONSTITUTION.md §108.p) + the §108 cluster header & ToC updated. Herald-applicability classification: APPLICABLE (integration in progress) — unlike the §108.o video non-applicability, Docs Chain directly governs Herald's ~76-doc html/pdf/docx sibling surface (migration plan `docs/research/docs_chain/HERALD_DOCS_CHAIN_PLAN.md`, planned E146 drift-gate); honest open prerequisites recorded (Phase-4 CLI landed 2026-05-31; exec-staging relative-asset gap G6; binary-hash verify defect found by dogfooding, fix in flight). Required by the upcoming Helix-side `CM-COVENANT-114-106-PROPAGATION` pre-build gate. Prior r12: added §110 (Intent recognition & clarification) restating the operator mandate (2026-05-31) at Herald project-constitution level — subscribers speak plain natural language (NO command syntax / no `COMMAND:` prefix); the three-tier intent-resolution discipline (Tier 1 deterministic CommandRecognizer fast-path → Tier 2 LLM intent inference via the `<<<HERALD-DISPATCH-v1>>>` envelope → Tier 3 `action="clarify"` reply-tag-and-ask fallback) as a table; the command set Tier 1 recognizes as a table; the "never guess / never ignore" rules; cites the authoritative contract `docs/design/INTENT_RECOGNITION.md` and notes inheritance from HelixConstitution §11.4.105 (the root-§ being added on the constitution stream) per §11.4.35; restated + cited, not redefined; ToC entry added. Prior r11: added §109 (Participant identity, attribution & notification-tagging) restating the operator mandate (2026-05-31) at Herald project-constitution level — per-messenger Participant identity (`subscribers` + `subscriber_aliases.username`), the `HERALD_<CHANNEL>_OPERATOR_USERNAME` operator env var (`HERALD_TGRAM_OPERATOR_USERNAME=@milos85vasic`), `created_by`/`assigned_to` attribution, and the @-tagging matrix (Claude +

Field	Value
	Operator never tagged) as a table; cites the authoritative contract `docs/design/PARTICIPANT_ATTRIBUTION.md` and notes inheritance from HelixConstitution per §11.4.35; restated + cited, not redefined; ToC entry added. Prior r10: extended §108 with §108.o restating inherited HelixConstitution §11.4.100 (Video color + visual-quality fidelity mandate, 2026-05-28) at Herald project-constitution level per §1.1 multi-file propagation discipline; inherited per §11.4.35, restated + cited, not redefined; literal anchor `11.4.100` now present in all three Herald root docs (CLAUDE.md r16, AGENTS.md r12, QWEN.md). Herald-applicability classification: non-applicable-but-cite (Herald has NO video-playback surface — pherald downloads video attachments as opaque sha256-blobs without decoding/rendering); cascade-parallel to §108.k (Universal §11.4.96 "Herald has no AOSP build, but the principle binds"). Required by upcoming Helix-side `CM-COVENANT-114-100-PROPAGATION` pre-build gate. Prior r7: extended §108 with §108.j–§108.l restating the next three inherited HelixConstitution mandates — §11.4.95 workable-items SQLite DB tracked-in-git-not-gitignored, §11.4.96 safe-parallel-work-with-long-build catalogue + mandate, §11.4.97 maximum-use-of-idle-time + progress-update cadence — at Herald project-constitution level per §1.1 multi-file propagation discipline; inherited per §11.4.35, restated + cited, not redefined; literal anchors `11.4.95`/`11.4.96`/`11.4.97` now present in all three Herald root docs. Prior r6: extended §108 with §108.d–§108.i restating the next six inherited HelixConstitution mandates — §11.4.89 background-test execution, §11.4.90 Obsolete status + obsolescence audit, §11.4.91 summary-doc clarity, §11.4.92 multi-pass change-evaluation, §11.4.93 SQLite workable-items SSoT, §11.4.94 zero-idle parallel-by-default — at Herald project-constitution level per §1.1 multi-file propagation discipline; inherited per §11.4.35, restated + cited, not redefined; literal anchors `11.4.89`–`11.4.94` now present. Prior r5: added §108 (with §108.a/b/c) restating the three inherited HelixConstitution mandates — §11.4.85 stress + chaos test mandate, §11.4.87 endless-loop autonomous work + zero-idle agent dispatch + anti-bluff testing, §11.4.88 background-push — at Herald project-constitution level per §1.1 multi-file propagation discipline; inherited per §11.4.35, restated + cited, not redefined. Required by the §11.4.87 `CM-COVENANT-114-87-PROPAGATION` pre-build gate. Prior r4: added §107 End-user-usability covenant (verbatim operator mandate restated at Herald level per §1.1 propagation) + paired gate invariant I8a–c; corrected stale "Owned-submodule set: (none)" to reflect the 10 vendored modules (9 Helix-stack + containers).
Issues	none
Issues summary	—
Fixed	R-14 (V2), V3-path-sync (V3 r3), §107 mandate + I8 gate invariant + owned-submodule list (r4), §108 Helix §11.4.85 + §11.4.87 + §11.4.88 propagation (r5), §108.d–§108.i Helix §11.4.89–§11.4.94 propagation (r6), §108.j–§108.l Helix §11.4.95–§11.4.97 propagation (r7)
Fixed summary	§106 spec-change rule retargeted to V3 path; gate-checked anchor 'comprehensive planning and implementation' unchanged so I7 stays green. r4 adds the end-user-usability covenant at Herald level (Helix §11.4 is the canonical authority; Herald restates per §1.1 multi-file propagation discipline) and the I8 gate invariant that asserts the mandate is present in CLAUDE.md + AGENTS.md + this file. r5 adds §108 (§108.a stress+chaos §11.4.85, §108.b endless-loop autonomous work §11.4.87, §108.c background-push §11.4.88) restating the three new inherited HelixConstitution mandates at Herald level per the §11.4.87 `CM-COVENANT-114-87-PROPAGATION` pre-build gate, inherited per §11.4.35;

Field	Value
	restated + cited, not redefined. r6 extends §108 with §108.d–§108.i (§108.d background-test execution §11.4.89, §108.e Obsolete status + obsolescence audit §11.4.90, §108.f summary-doc clarity §11.4.91, §108.g multi-pass change-evaluation §11.4.92, §108.h SQLite workable-items SSoT §11.4.93, §108.i zero-idle parallel-by-default §11.4.94) restating the next six inherited HelixConstitution mandates at Herald level, inherited per §11.4.35; restated + cited, not redefined; literal anchors `11.4.89`/`11.4.90`/`11.4.91`/`11.4.92`/`11.4.93`/`11.4.94` now present in all three Herald root docs. r7 extends §108 with §108.j–§108.l (§108.j workable-items SQLite DB tracked-in-git-not-gitignored §11.4.95, §108.k safe-parallel-work-with-long-build catalogue + mandate §11.4.96, §108.l maximum-use-of-idle-time + progress-update cadence §11.4.97) restating the next three inherited HelixConstitution mandates at Herald level, inherited per §11.4.35; restated + cited, not redefined; literal anchors `11.4.95`/`11.4.96`/`11.4.97` now present in all three Herald root docs.

Continuation —

Table of contents

This constitution **extends** the Helix Universal Constitution provided by the **parent project's** `constitution/` submodule. Herald does not carry its own copy — see `docs/guides/CONSTITUTION_INHERITANCE.md` for the discovery contract.

All clauses in the parent-provided `constitution/Constitution.md` apply unless explicitly overridden below with an explicit `Override §X.Y` section.

Canonical constitution repo: https://github.com/HelixDevelopment/HelixConstitution

Project Articles

§101. Pre-implementation status

Herald is pre-implementation. Until a `go.mod` is committed, no clause below may be interpreted as authorizing the agent to fabricate build/test infrastructure that doesn't yet exist. Confirm the disambiguation (scaffold vs. fill spec) with the operator before writing code, per Universal §11.4.6 (no-guessing).

§102. Mission boundary

Herald ingests system events and fans them out to multiple notification channels. Anything outside event ingestion or notification fan-out is **out of scope** for this repo and belongs in a different submodule of the consuming project.

§103. Mirror parity (extends Universal §2.1)

Every commit on `main` MUST land on all four upstream hosts (GitHub, GitLab, GitFlic, GitVerse) in a single fan-out push. The repo's `origin` remote already aggregates the four push URLs; do not bypass `origin` with per-host pushes unless rebuilding the fan-out configuration.

§104. No embedded constitution (extends Universal §3)

Herald **MUST NOT** carry its own `constitution/` submodule. Herald is consumed as a submodule of a parent project that already provides the constitution; a duplicated copy would diverge in pinning, cause confusion about which is authoritative, and violate the "submodule commits propagate first" propagation order (Universal §3). The inheritance gate's invariant `I6` enforces this: it FAILs if `<repo-root>/constitution/` or `.gitmodules` reappears.

For standalone development of Herald (no parent project), clone the constitution **alongside** Herald, not inside it:

```
git clone git@github.com:HelixDevelopment/HelixConstitution.git \
    $(dirname "$PWD")/constitution
```

§105. Inheritance gate (extends Universal §1.1)

`tests/test_constitution_inheritance.sh` discovers the parent-provided constitution via inline parent-walk (mirrors `find_constitution.sh` Phase 1), then asserts six invariants:

#	Invariant
I1	A `constitution/Constitution.md` is reachable by walking up the parent chain from Herald's repo root.
I2	The discovered `Constitution.md` contains the `§11.4 End-user quality guarantee — forensic anchor` line (the exact string the §1.1 mutation removes).
I3	The discovered `CLAUDE.md` contains the `MANDATORY ANTI-BLUFF COVENANT` anchor.
I4	The discovered `AGENTS.md` contains the `Anti-bluff covenant` anchor.
I5a–d	Herald's root docs (`CLAUDE.md`, `AGENTS.md`, this file, and `README.md`) all declare the parent-discovery inheritance contract.

#	Invariant
I6	No `constitution/` directory or `.gitmodules` file exists at Herald's root (the §104 invariant).
I7a–c	Herald's `CLAUDE.md`, `AGENTS.md`, and this file all contain the §106 spec-change rule anchor (per §1.1 propagation, mutation-paired).
I8a–c	Herald's `CLAUDE.md`, `AGENTS.md`, and this file all contain the §107 end-user-usability covenant anchor (the verbatim operator-mandate quote — per §1.1 propagation, mutation-paired).

`tests/test_constitution_inheritance_meta.sh` delegates to the discovered constitution's `meta_test_inheritance.sh`, which strips the §11.4 anchor from `Constitution.md`, runs the gate, and asserts FAIL — proving the gate is not a bluff (Universal §1.1).

Both scripts run as a precondition to any commit that touches root docs or the discovery contract.

§106. Spec-change rule (extends Universal §11.4)

Any modification to `docs/specs/mvp/specification.V4.md` or any file under `docs/specs/` (any depth) triggers **mandatory comprehensive planning and implementation of all changes**. An agent or contributor may not edit the spec in isolation: every change is a project-wide ripple that requires the corresponding code, tests, and downstream doc updates in the same logical work effort.

This rule does NOT apply to creating or renaming files; for those, the operator must explicitly tell the worker (CLI agent or human contributor) what to do with the newly created or copied files.

Propagation. The same rule is restated in Herald's `CLAUDE.md` and `AGENTS.md` (per §1.1 multi-file propagation discipline). The inheritance gate's invariant **I7a–c** asserts the rule's anchor literal (`comprehensive planning and implementation`) is present in all three files; a missing copy is a propagation bluff and the gate FAILs.

Anchor (forensic): the literal text `Whenever this document (\docs/specs/mvp/specification.V4.md`)MUST appear indocs/specs/mvp/specification.V4.md` §"Specification documents" — that line is the source of truth that all three propagated copies summarize.

Paired §1.1 mutation (planned). Removing the spec-change anchor from any of the three propagation files MUST cause `I7a/b/c` to FAIL; the paired meta-test will be added when `test_constitution_inheritance_meta.sh` is generalised beyond its current single-anchor mutation.

§107. End-user-usability covenant (extends Universal §11.4 — MANDATORY ANTI-BLUFF)

Forensic anchor — verbatim operator mandate (first declared 2026-04-28, reasserted 2026-05-19 and 2026-05-20):

> "all existing tests and Challenges do work in anti-bluff manner - they MUST confirm that all tested codebase really works as expected! We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can't be used! This MUST NOT be the case and execution of tests and Challenges MUST guarantee the quality, the completition and full usability by end users of the product! This MUST BE part of Constitution of our

> project, its CLAUDE.MD and AGENTS.MD if it is not there already, and to be applied to all Submodules's Constitution, CLAUDE.MD and AGENTS.MD as well (if not there already)!"

Canonical authority. Helix Universal Constitution §11.4 + §11.4.1..§11.4.16, restated at the parent's `constitution/CLAUDE.md` "MANDATORY ANTI-BLUFF COVENANT — END-USER QUALITY GUARANTEE" section and at the parent's `constitution/AGENTS.md` matching section. Herald inherits the covenant unconditionally; §107 exists to make the restatement explicit at Herald level and to bind every Herald binary (`pherald`, `sherald`, `cherald`, `bherald`, `rherald`, `iherald`, `scherald`, …) to the covenant on its own terms.

Operative rule (Herald-binding).

1. The bar for shipping any Herald feature is **NOT** "tests pass" or "the binary compiles" — it is **"the end user of `<flavor>herald` can actually use the feature."**
2. Every PASS — unit test, integration test, gate, Challenge, smoke test, e2e — MUST carry positive runtime evidence that the user-visible behaviour works. Metadata-only PASS, configuration-only PASS, "absence-of-error" PASS, and grep-only PASS without runtime evidence are §11.4 PASS-bluffs and constitute critical defects regardless of how green the summary line looks.
3. Tests AND Challenges are bound **equally**. A Challenge that scores PASS on a non-functional feature is the same class of defect as a unit test that does.
4. The canonical Herald enforcement is `scripts/e2e_bluff_hunt.sh` — it builds `pherald`, runs the full test suite, starts a real Gin server, hits every `/v1/*` route, asserts response bodies, boots a real Postgres container via the `containers/` submodule, runs the M2 integration tests against the live DB, and SIGTERM-graceful-shutdowns. A single FAIL invariant means a real feature is broken for end users; no release tag, no risky commit, and no "implementation milestone landed" claim may be made while it FAILs.
5. New user-visible Herald features (V3 §§11, 33, 41, 42, 43 and beyond) MUST extend `e2e_bluff_hunt.sh` with a new `E_N` invariant in the same logical work effort — a feature that ships without its e2e invariant is shipping without anti-bluff evidence and violates §107.

Propagation. This §107 is restated verbatim (in summary form, citing this section as the canonical Herald source) in Herald's `CLAUDE.md` and `AGENTS.md` per §1.1 multi-file propagation discipline. The inheritance gate's invariant `I8a-c` asserts the anchor literal (`End-user-usability covenant` or the verbatim "all tests do execute with success" operator quote) is present in all three files; a missing copy is a propagation bluff and the gate FAILs.

Paired §1.1 mutation. A future generalised mutation-meta will assert that removing the §107 anchor from any of the three propagation files MUST cause `I8a/b/c` to FAIL — the §1.1 paired-mutation discipline is non-negotiable for every new gate.

Non-compliance is a release blocker. No flavor binary may be tagged, no submodule may be propagated, and no operator-handoff (`docs/CONTINUATION.md`) may be published while a §107 evidence-gap is open.

§107.x. docs/qa/ Evidence Mandate (operator mandate, 2026-05-22 — extends §107; cascades from Helix §11.4.83)

Forensic anchor — verbatim operator mandate (2026-05-22):

> "every feature that ships MUST carry a recorded e2e communication transcript + any attached materials under `docs/qa/<run-id>/` (per-feature subdirectories). A

feature with no QA transcript is itself a §107 PASS-bluff — it claims to work but has no auditable runtime evidence. Bot-driven automation (e.g. Herald's planned `qaherald` binary) MUST preserve full bidirectional communication threads as proof."

Canonical authority. Helix Universal Constitution §11.4.83 (the rule was added at universal level in the same propagation cycle as this §107.x). Herald §107.x is the project-binding restatement.

Operative rule (Herald-binding).

1. Every Herald feature that ships — every flavor binary route (`/v1/events`, `/v1/compliance`, `/v1/safety_state`, …), every Telegram bot interaction (HRD-011), every Claude Code dispatch path (HRD-012), every container-orchestrated flow, every QA-bot transcript (planned `qaherald` per HRD-NNN-to-be-assigned) — MUST carry a `docs/qa/<run-id>/` directory committed in the same logical work effort (per V3 §8.3 HRD lifecycle). `<run-id>` is monotonic + greppable: `HRD-NNN`, `HRD-NNN-<TS>` (multi-run), or a free `<TS>` tag.
2. Transcripts are **full bidirectional** — for Telegram: both user→bot and bot→user halves; for Gin: both request payload (JSON or TOON per Wave 4b) and full response body + status line + relevant headers; for Claude Code: both the prompt sent and the response received; for container flows: container stdin → container stdout/stderr + container logs.
3. Attached materials are committed **in-repo**, never linked. Screenshots in `.png`; JSON / TOON payloads as their natural extension; container logs as `.log`; OpenTelemetry trace exports as `.json` / `.otlp`. External-only links (Slack URL, Drive URL, Telegram message URL) are §11.4.13 sink-side violations — the artefact lives in `docs/qa/<run-id>/`.
4. The planned `qaherald` binary (HRD-NNN to be assigned) is Herald's authoritative QA automation. It drives `pherald` ⍰ Telegram (and analogous round-trips for the other flavors) and preserves the full conversation thread under `docs/qa/qaherald-<TS>/`. A `qaherald` run that stores only the final PASS/FAIL line is itself a §107.x bluff at the QA-automation layer.
5. New e2e_bluff_hunt invariants for features added after 2026-05-22 MUST cite their `docs/qa/<run-id>/` artefact as positive-evidence anchor (composes with §11.4.2 / §11.4.5). A new `E_N` invariant without a corresponding `docs/qa/` directory is a §107.x violation in the same logical work effort.
6. Release-gate enforcement: `scripts/release.sh` (when implemented) + the existing tag-time guard MUST refuse to tag a Herald release whose feature-shipping commits since the previous tag lack their matching `docs/qa/<run-id>/` directories.

Propagation. §107.x is restated (in summary form, citing this section as the canonical Herald source) in Herald's `CLAUDE.md`, `AGENTS.md`, `QWEN.md`. The universal mandate at Helix §11.4.83 is cascaded into the 11 Helix-stack submodules (`auth`, `background`, `cache`, `Concurrency`, `config`, `database`, `eventbus`, `middleware`, `Models`, `observability`, `recovery`) per §1.1 multi-file propagation discipline.

Non-compliance is a release blocker. No `--qa-evidence-optional`, `--qa-transcript-later`, `--qa-bot-summary-suffices` flag exists.

§107.y. Working-Tree Quiescence Rule (operator mandate, 2026-05-22 — extends §107; cascades from Helix §11.4.84)

Short tag: `working-tree quiescence`.

Forensic anchor — verbatim operator mandate (2026-05-22):

> "no subagent commit may proceed while any concurrent mutation gate is in flight in the same checkout. Before `git add`, the committing agent MUST `grep` its own working tree for mutation markers (`MUTATED for paired`, `// always pass`, `return json.Marshal` shortcut paths, etc.). Any unexplained file in the staging area triggers ABORT."

Canonical authority. Helix Universal Constitution §11.4.84. Herald §107.y is the project-binding restatement.

Lesson (forensic case study — Herald-internal, 2026-05-21). A logo-fix subagent (commit `72e81ab`, "Fix: replace pandoc {width=96px} image attr with HTML tag") ran in this very checkout while a paired §1.1 Wave 4b mutation gate had temporarily injected an `// always pass` shortcut into `commons_auth/middleware.go` (JWT-bypass mutation, intended for the mutate → assert FAIL → restore cycle). The subagent's `git add` swept the mutation residue into its commit; the commit was pushed to all four mirrors before any other agent caught it. Within the hour the SECURITY FIX (`d5bd360`, "SECURITY FIX: restore commons_auth/middleware.go JWT verify (mutation residue in 72e81ab)") restored the verify path. But the production-equivalent-binary-with-bypassed-JWT window is a real security-defect window — small, but non-zero, and demonstrably exploitable in that interval. The rule below is the constitutional outcome. This is no longer a hypothetical; it is documented Herald history.

Operative rule (Herald-binding).

1. **Pre-`git add` quiescence check.** Every commit flow (main thread + every dispatched subagent) MUST grep the working tree for the canonical Herald mutation markers BEFORE `git add`:
   - `MUTATED for paired` (the canonical paired-§1.1 marker emitted by `tests/test_wave4b_mutation_meta.sh` + future generalisations)
   - `// always pass`, `// MUTATION`, `# MUTATION` (Go + shell mutation annotations)
   - `return json.Marshal` shortcut paths in `commons/` or `commons_messaging/` (Wave 4b TOON mutation residue)
   - `_mutated_*` filename suffixes
   - `.git/MUTATION_IN_PROGRESS` (the lockfile)
2. **Scope-match.** Cross-check `git status --porcelain` against the subagent's declared scope. Any file outside the declared scope → ABORT. The subagent MUST explicitly account for every modified / untracked / staged entry.
3. **Lockfile serialisation.** When any mutation gate is in flight, its first action is `touch .git/MUTATION_IN_PROGRESS`; its last action (trap-on-exit) is `rm .git/MUTATION_IN_PROGRESS`. Any subagent finding this lockfile present MUST refuse to `git add` and ABORT until the gate completes its mutate → assert FAIL → restore → assert PASS cycle and removes the lockfile.
4. **Worktree isolation (preferred).** When parallel subagents are required (§11.4.20 / §11.4.70 subagent-driven default), prefer `git worktree add` per subagent over single-checkout concurrency — eliminates the cross-mutation race by construction.
5. **Pre-push mutation-residue scanner.** `scripts/mutation_residue_audit.sh` (planned, HRD-NNN to be assigned) MUST run before every push. Any commit in the pushed range containing a mutation marker → push BLOCKED.

Prototype. `tests/test_wave4b_mutation_meta.sh` ALREADY includes the canonical Herald implementation:

- `check_quiescence()` at line 92 — the working-tree quiescence guard (returns 0 if NO MUTATED markers in tracked files).
- Line 197 — the "Working-tree quiescence — assert no MUTATED markers leaked" assertion.

The Wave 4b test is the prototype; generalising the check across every paired-§1.1 gate (Wave 2, Wave 3, Wave 4a) is open work. The planned universal scanner `scripts/mutation_residue_audit.sh` is the roll-out vehicle.

Composes with §107 (a security-bypass mutation that ships to production is the gravest §107 PASS-bluff), §1.1 (paired-mutation discipline — the rule protects the mutation cycle from concurrent contamination), §11.4.20 / §11.4.70 (subagent-driven default — quiescence rule makes parallel subagent dispatch safe), §11.4.10 (credentials handling — same class of "no unrelated content in a commit"), §11.4.27 (no-fakes-beyond-unit — a mutation residue swept into a commit IS a fake-pass surface in production).

Propagation. §107.y is restated (in summary form, citing this section as the canonical Herald source) in Herald's `CLAUDE.md`, `AGENTS.md`, `QWEN.md`. The universal mandate at Helix §11.4.84 is cascaded into the 11 Helix-stack submodules per §1.1 multi-file propagation discipline.

Non-compliance is a release blocker. A mutation marker that lands in a tagged Herald commit is a critical defect regardless of how briefly it persisted — see commits `72e81ab` / `d5bd360` as forensic proof. No `--allow-residue`, `--skip-quiescence`, `--mutation-cleanup-later` flag exists.

§108. Inherited covenant restatements (Helix §11.4.85 / §11.4.87 / §11.4.88 / §11.4.89 / §11.4.90 / §11.4.91 / §11.4.92 / §11.4.93 / §11.4.94 / §11.4.95 / §11.4.96 / §11.4.97 / §11.4.98 / §11.4.99 / §11.4.100 / §11.4.106)

These twelve mandates are **inherited** from the Helix Universal Constitution via parent-discovery (§11.4.35). Herald **restates + cites** them at project-constitution level per the §1.1 multi-file propagation discipline — it does **NOT** redefine, narrow, or weaken them; the parent `constitution/Constitution.md` is the canonical authority for each. The literal anchors (`11.4.85`, `11.4.87`, `11.4.88`, `11.4.89`, `11.4.90`, `11.4.91`, `11.4.92`, `11.4.93`, `11.4.94`, `11.4.95`, `11.4.96`, `11.4.97`) are required across Herald's `CLAUDE.md` / `AGENTS.md` / `QWEN.md` by the §11.4.87 `CM-COVENANT-114-87-PROPAGATION` pre-build gate, which strips the literal in a paired §1.1 meta-test mutation and asserts the gate FAILs.

§108.a. Stress + Chaos Test Mandate (extends Universal §11.4.85; Helix 2026-05-24)

Forensic anchor — verbatim user mandate (2026-05-24):

> "Every fix or improvement you do MUST BE covered with full automation stress and chaos tests so we are sure nothing can break the functionality and all edge cases are monitored and polished and additionally fixed if that is needed! Everything must produce rock solid proofs and follow fully no-bluff policy!"

Canonical authority. Helix Universal Constitution §11.4.85. Herald §108.a is the project-binding restatement.

Operative rule (Herald-binding). Every fix or improvement landed in Herald MUST ship with full-automation **stress** AND **chaos** test suites. Stress = sustained load (N ≥ 100 iterations or ≥ 30 s, p50/p95/p99 recorded) and/or concurrent contention (N ≥ 10 parallel, no deadlock / leak / data race) and/or boundary conditions (empty / max / off-by-one, each producing a categorised result). Chaos = at least one failure-injection category appropriate to the fix-class: process-death, network-fault, input-corruption, resource-exhaustion (disk-full / OOM / fd-exhaustion), or state-corruption (DB-lock-loss / partial-write / cache-invalidation). Every stress + chaos PASS cites a captured-evidence artefact under `docs/qa/<run-id>/stress_chaos/` per §11.4.5 + §11.4.69 (latency.json / throughput.csv / categorised_errors.txt / recovery_trace.log); chaos-injection cleanup is non-negotiable (`trap '...' EXIT` restores corrupted `.env`, `rm`s disk-fillers, verifies killed processes restart). For Herald this binds the flavor binaries: `pherald listen` inbound long-poll under concurrent Telegram updates, Gin `/v1/*` routes under sustained load, claude_code dispatch under process-death, container-orchestrated flows under disk/OOM pressure. A happy-path-only PASS is a §107 / §11.4 PASS-bluff at the resilience layer. No `--skip-stress` / `--no-chaos` / `--happy-path-suffices` / `--stress-test-later` escape exists.

§108.b. Endless-loop autonomous work + zero-idle agent dispatch + anti-bluff testing (extends Universal §11.4.87; Helix 2026-05-26)

Forensic anchor — verbatim user mandate (2026-05-26, condensed):

> "all work MUST BE continued in endless loop until there is no any open items, no unfinished workable items from our Issues docs, or from Continuation document or any unfinished work by agents … fully autonomously. You will spawn agents or agents-driven work whenever that is possible or required! Not a single agent or main work stream will sit idle except if it waits for the results of something … All work MUST BE always covered with comprehensive tests … which produce real proofs … and in complete anti-bluff manner!"

Canonical authority. Helix Universal Constitution §11.4.87. Herald §108.b is the project-binding restatement.

Operative rule (Herald-binding). When the operator instructs Herald work to "continue in endless loop fully autonomously" (or equivalent), it is a HARD-CONTRACT covenant: (A) **continuation** — continue until ALL are simultaneously TRUE: Herald's autonomous loop checks `docs/Issues.md` Status-column has zero `In progress` / `Ready for testing` / `In testing` / `Reopened` entries (§11.4.15 closed-set), `docs/CONTINUATION.md` §3 "Active work" is empty, the TaskList reports no subagent mid-execution, and no in-flight external dependency (build, push, sync) remains; (B) **zero-idle dispatch** — dispatch background subagents for parallelisable, non-file-scope-contending work rather than serialising; idle is permitted ONLY while waiting on a result; (C) **comprehensive test coverage with real (physical) proofs** — every closed item lands four-layer coverage per §11.4.4(b) with captured-evidence PASS; (D) **anti-bluff end-to-end** — tests AND Challenges are bound equally (a Challenge that scores PASS on a non-functional feature is the same defect class as a unit test that does); (E) **termination** — only on all-clear, an explicit operator `STOP` / `END LOOP`, a §12 host-session-safety demand, or a scheduled wake against a known-future-actionable signal. Composes with §107 (anti-bluff) and §12.10 (CONTINUATION-doc maintenance — the source-of-truth state the loop checks). No `--idle-OK` / `--skip-endless-loop` / `--bluff-permitted-for-this-task` / `--metadata-only-test-suffices` / `--no-physical-proof-required` escape exists.

§108.c. Background-push mandate: commit-lock release immediately after commit, push runs detached (extends Universal §11.4.88; Helix 2026-05-26)

Forensic anchor — verbatim user mandate (2026-05-26, condensed):

> "Make sure all these pushes are being done ALWAYS in background in parallel with main work stream so we do not loose time waiting. Everything is committed anyway? … We MUST ensure that main work stream has always something to do, or wait for the results only when that is absolutely required!"

Canonical authority. Helix Universal Constitution §11.4.88. Herald §108.c is the project-binding restatement.

Operative rule (Herald-binding). (A) **Flock release immediately after commit** — once `git commit` returns 0 (commit object durable on local disk), the `.git/.commit_all.lock` flock MUST be released BEFORE any push runs; gating further local work on a remote round-trip is the exact zero-idle anti-pattern §108.b / §11.4.87 prohibits. (B) **Push runs detached** — `nohup ./push_all.sh ... > <log> 2>&1 &` then `disown`; the orchestrator's exit code reports COMMIT success, not push success. (C) **Per-remote serialisation, multi-remote parallelism** — `push_all.sh` acquires a per-remote flock (`.git/.push.<remote>.lock`) so same-remote invocations serialise (no non-fast-forward race) while Herald's four mirrors (GitHub / GitLab / GitFlic / GitVerse) push in parallel. (D) **Failure surface** — backgrounded push failures land in `qa-results/push_failures/<TS>_<remote>.log`; the next autonomous-loop tick MUST check that directory (the §108.b(A) "no external dependency in-flight" check) and surface failures — silent push-failure is a §11.4 PASS-bluff at the distribution layer. (E) **Synchronous-push escape** — the explicit `--sync-push` flag preserves legacy synchronous behaviour for §11.4.41 force-push merge-first paths; it is the ONLY escape. Composes with §103 (mirror parity — §108.c makes the four-mirror fan-out non-blocking) and §108.b (this is the implementation seam that makes the endless loop genuinely zero-idle for the dominant blocker class).

Propagation. §108 (with §108.a/b/c) is restated (in summary form, citing this section as the canonical Herald source) in Herald's `CLAUDE.md`, `AGENTS.md`, `QWEN.md`. The universal mandates at Helix §11.4.85 / §11.4.87 / §11.4.88 are cascaded into the Helix-stack submodules per §1.1 multi-file propagation discipline. The §11.4.87 `CM-COVENANT-114-87-PROPAGATION` pre-build gate enforces the literal anchors `11.4.85` / `11.4.87` / `11.4.88` in all three Herald root docs.

Non-compliance is a release blocker. Each sub-clause carries its inherited no-escape-hatch posture: no `--skip-stress` / `--no-chaos` (§108.a), no `--idle-OK` / `--skip-endless-loop` (§108.b), no synchronous push without `--sync-push` (§108.c).

§108.d. Background test execution mandate (extends Universal §11.4.89; Helix 2026-05-27)

Forensic anchor — verbatim user mandate (2026-05-27, condensed):

> "long-running tests (the mutation gates, e2e bluff-hunt, stress and chaos suites) MUST run in background so the main work stream never blocks waiting on them — survey the results from the log when they finish, do not sit idle."

Canonical authority. Helix Universal Constitution §11.4.89. Herald §108.d is the project-binding restatement.

Operative rule (Herald-binding). Any Herald test cycle expected to exceed ~30s — the `tests/test_wave*_mutation_meta.sh` gates, `scripts/e2e_bluff_hunt.sh`, future stress/chaos suites — MUST run detached (`nohup … > qa-results/<test_id>_<TS>.log 2>&1 &` then `disown`); the main work stream returns immediately to the §11.4.42 priority queue and polls the log / exit-status rather than blocking. Composes with §107.y / §11.4.84: a backgrounded mutation gate still mutates the shared tree, so only ONE runs against the main checkout at a time (serialised by the `.git/MUTATION_IN_PROGRESS` lockfile + the planned `scripts/mutation_residue_audit.sh` pre-push scanner); concurrent gates require separate `git worktree` checkouts. Foreground is permitted only for <30s tests or on explicit operator request. No `--block-on-long-test` / `--skip-background` escape exists.

§108.e. Obsolete status + per-item obsolescence audit (extends Universal §11.4.90; Helix 2026-05-27)

Forensic anchor — verbatim user mandate (2026-05-27, condensed):

> "some workable items are no longer valid — superseded by a later design or mandate, or the feature was removed. Add an Obsolete status and audit every item for obsolescence at each release, with triple-checked evidence for why it is obsolete — no bare assertions."

Canonical authority. Helix Universal Constitution §11.4.90. Herald §108.e is the project-binding restatement.

Operative rule (Herald-binding). Herald's HRD Status closed-set gains a 4th terminal value `Obsolete (→ Fixed.md)` for items no longer valid (Reason closed-set: superseded-by-design-change / superseded-by-later-mandate / feature-removed / duplicate-of / unsupported-topology). Every `Obsolete` HRD heading MUST carry an `**Obsolete-Details:**` line within 8 non-blank lines of the heading: Since (ISO date), Reason (closed-set value), Superseding-item (§ or HRD reference), and Triple-check evidence (git-log / grep / runtime path per §11.4.6 — a bare assertion is forbidden). At every release-gate sweep, Herald re-evaluates every open + Fixed HRD for obsolescence; Obsolete→Fixed.md migrations are atomic per §11.4.19 (the §107.x docs/qa evidence mandate still applies where the obsoleted item once shipped a feature). No `--obsolete-without-evidence` / `--skip-obsolescence-audit` escape exists.

§108.f. Summary-doc clarity (extends Universal §11.4.91; Helix 2026-05-27)

Forensic anchor — verbatim user mandate (2026-05-27, condensed):

> "the one-line summaries in the Issues/Fixed/Status summary docs are useless when they are just a fragment or a status word — each MUST be a self-contained clause that says what the item is about and what the problem or goal is."

Canonical authority. Helix Universal Constitution §11.4.91. Herald §108.f is the project-binding restatement.

Operative rule (Herald-binding). Every one-liner in `docs/Issues_Summary.md` / `docs/Fixed_Summary.md` / `docs/Status_Summary.md` and every README doc-link row MUST be a self-contained clause (≥6 words OR ≥40 chars) naming the SUBJECT + the PROBLEM/GOAL — never a section-label fragment (`Composes with`, `Closure criteria`), bare metadata (`Critical`, `Bug`), a status restatement, or a bare HRD-id. Each is derived from the source long-form H1/H2 heading, never invented. The `*_Summary.md`

variants remain derived (per the documentation-artefacts rule — do not hand-edit them out of sync with source). No `--terse-summary-OK` escape exists.

§108.g. Multi-pass change-evaluation discipline (extends Universal §11.4.92; Helix 2026-05-27)

Forensic anchor — verbatim user mandate (2026-05-27, condensed):

> "before a change is ready to commit, evaluate it in multiple passes — the main task, the regression blast-radius, the cross-feature interactions, the deep research, and a final anti-bluff confirmation — and document each pass. A change that only proved its happy path is not done."

Canonical authority. Helix Universal Constitution §11.4.92. Herald §108.g is the project-binding restatement.

Operative rule (Herald-binding). Every non-trivial Herald change MUST pass — and document — a 5-pass evaluation before it is commit-ready: **Pass 1** main-task captured-evidence (§11.4.5 / §107 — no "should work"); **Pass 2** regression-blast-radius (every touched file + every importer/caller audited — e.g. a `commons` type change audits all flavor binaries); **Pass 3** cross-feature interaction (shared state / timing / env — e.g. a mutation-gate edit checks §107.y quiescence + §108.d / §11.4.89 backgrounding); **Pass 4** deep-research validation (§11.4.8 — external precedent located, or a literal "NO external solution found" recorded); **Pass 5** anti-bluff confirmation (no metadata-only / config-only / script-bug PASS). Evidence lands in the commit footer or under `docs/qa/` / `qa-results/`. Trivial changes (typo, revision-bump, MD-export regen touching zero source) are exempt only with an explicit commit-message citation. No `--single-pass-OK` / `--skip-blast-radius` escape exists.

§108.h. SQLite-backed single-source-of-truth for workable items (extends Universal §11.4.93; Helix 2026-05-27)

Forensic anchor — verbatim user mandate (2026-05-27, condensed):

> "the Issues / Fixed / Status / Continuation tracking is in too many Markdown files that drift out of sync — back it with a single SQLite source of truth and regenerate the Markdown from it (and back) so drift is impossible. The tool already exists in the constitution — use it, do not reinvent it."

Canonical authority. Helix Universal Constitution §11.4.93. Herald §108.h is the project-binding restatement.

Operative rule (Herald-binding). Herald's text-based HRD trackers (`docs/Issues.md` / `docs/Fixed.md` / the `*_Summary.md` variants / `docs/Status.md` / `docs/CONTINUATION.md` §3) migrate to a SQLite single-source-of-truth (`docs/.workable_items.db`, **version-controlled as Herald's authoritative SSoT** — operator mandate 2026-05-27: "We should not git ignore our workable items database since it is our single source of truth regarding items we have to do on the project!". This is a deliberate Herald divergence from the parent §11.4.93 gitignored-with-regeneration default: for Herald the DB IS the authoritative artefact, so it is committed and version-controlled, NOT regenerated-on-clone; only the transient SQLite WAL/SHM sidecars (`docs/.workable_items.db-wal` / `-shm`) stay gitignored) with bidirectional MD⇄DB regeneration so sync-drift is mechanically impossible. The migration Go binary lives in the constitution submodule (`constitution/scripts/workable-items/`) — Herald references it per §11.4.74 catalogue-first, and never reimplements it. The migration is 6-phase; Herald files the tracking HRD (Phase 1) and

progresses incrementally without breaking the §106 spec-change / §107.x evidence invariants. No `--skip-ssot` / `--keep-md-only` escape exists once the migration HRD is filed.

§108.i. Zero-idle priority-first parallel-by-default operating mode (extends Universal §11.4.94; Helix 2026-05-27)

Forensic anchor — verbatim user mandate (2026-05-27, condensed):

> "never sit idle while there is priority-queued work that could move — survey the queue, pick the highest-priority non-blocking items, and run them in parallel by default. Idle is only allowed when everything is genuinely blocked on something external."

Canonical authority. Helix Universal Constitution §11.4.94. Herald §108.i is the project-binding restatement.

Operative rule (Herald-binding). Binding always-on contract: Herald work is NEVER idle while a priority-queued item can progress. Before any wake / sleep / "waiting for X", survey the priority queue, identify all non-contending items, and dispatch them in parallel — subagent-driven per §11.4.20 / §11.4.70 when non-trivial, backgrounded per §108.d / §11.4.89 when >30s — picking the highest-Severity / priority item first. The conductor remains the integration + commit + push seam; parallel work MUST NOT compromise stability (composes with §107.y quiescence + §108.g / §11.4.92 multi-pass + §12 host-safety). This is the operating-mode generalisation of §108.b / §11.4.87 (endless-loop) and §108.c / §11.4.88 (background-push). Idle is permitted ONLY when every item is genuinely blocked on an external dependency, the operator issued `STOP`, or §12 host-session-safety demands it. No `--serialise-OK` / `--idle-permitted` escape exists.

Propagation (§108.d–§108.i). These six restatements are summarised — citing this section as the canonical Herald source — in Herald's `CLAUDE.md` and `AGENTS.md` (and `QWEN.md` when present). The universal mandates at Helix §11.4.89 / §11.4.90 / §11.4.91 / §11.4.92 / §11.4.93 / §11.4.94 are cascaded into the Helix-stack submodules per §1.1 multi-file propagation discipline. The literal anchors `11.4.89` / `11.4.90` / `11.4.91` / `11.4.92` / `11.4.93` / `11.4.94` are present in all three Herald root docs.

Non-compliance is a release blocker (§108.d–§108.i). Each sub-clause carries its inherited no-escape-hatch posture: no `--block-on-long-test` (§108.d), no `--obsolete-without-evidence` (§108.e), no `--terse-summary-OK` (§108.f), no `--single-pass-OK` (§108.g), no `--keep-md-only` once migration is filed (§108.h), no `--idle-permitted` (§108.i).

§108.j. Workable-items SQLite DB is TRACKED in git, never gitignored (extends Universal §11.4.95; Helix 2026-05-27)

Forensic anchor — verbatim user mandate (2026-05-27, condensed):

> "We should not git ignore our workable items database since it is our single source of truth regarding items we have to do on the project! — it must be committed and pushed, never regenerated-on-clone, and never silently rewritten without a backup."

Canonical authority. Helix Universal Constitution §11.4.95. Herald §108.j is the project-binding restatement.

Operative rule (Herald-binding). Herald ALREADY complies (operator correction 2026-05-27, recorded in §108.h + `.gitignore`): the workable-items SQLite DB is version-controlled, committed + pushed alongside every state change, WAL-checkpointed (`PRAGMA wal_checkpoint(TRUNCATE)`) before commit so only the transient SQLite `-wal`/`-shm` sidecars stay gitignored, and never force-rewritten without a §9.2 hardlinked-backup. This is an explicit §11.4.30 carve-out and AMENDS the earlier §11.4.93 / §108.h "gitignored-with-regeneration" text (which Herald had already diverged from per the same operator mandate). **Herald alignment note:** the constitution's canonical path is `docs/workable_items.db` (no leading dot); Herald's HRD-131 currently references `docs/.workable_items.db` — reconcile to the canonical path when the DB is implemented (HRD-131 Phase 2+, currently deferred). No `--gitignore-db` / `--regenerate-on-clone` escape exists.

§108.k. Safe-parallel-work-with-long-build catalogue + mandate (extends Universal §11.4.96; Helix 2026-05-27)

Forensic anchor — verbatim user mandate (2026-05-27, condensed):

> "while a long build or long-running operation is going, do not sit idle — there is a whole catalogue of work that is safe to do in parallel, so consult it and dispatch the safe non-contending items, and only avoid the things that would collide with the running operation."

Canonical authority. Helix Universal Constitution §11.4.96. Herald §108.k is the project-binding restatement.

Operative rule (Herald-binding). Herald has no AOSP build, but the principle binds: during ANY long-running Herald operation (a backgrounded mutation gate, `scripts/e2e_bluff_hunt.sh`, a §108.a / §11.4.85 stress/chaos suite, a doc export) the conductor MUST consult the safe-parallel catalogue and dispatch non-contending work rather than idle. SAFE-in-parallel for Herald: (A) MD/docs work, (B) `scripts/` helpers, (C) gate authoring, (D) test authoring, (E/F) commit + push to mirrors, (H) read-only analysis subagents, (I) web research, (J) workable-items DB ops. UNSAFE-during-a-running-gate (maps to §107.y / §11.4.84): `git checkout` / `reset --hard` / `clean` on files a gate is transiently mutating, a SECOND concurrent mutation gate against the same checkout (composes with §108.d / §11.4.89 single-gate serialisation), host-session-safety breaches (§12). Subagent-driven default per §11.4.20 / §11.4.70. This is the catalogue that makes §108.i / §11.4.94 (zero-idle parallel-by-default) concrete during Herald's longest-running operations. No `--idle-during-build` / `--skip-parallel-catalogue` escape exists.

§108.l. Maximum-use-of-idle-time mandate + progress-update cadence (extends Universal §11.4.97; Helix 2026-05-27)

Forensic anchor — verbatim user mandate (2026-05-27, condensed):

> "use every minute of idle time to move work forward — and keep me posted with short progress updates at each milestone without me having to ask, always backed by real captured evidence for anything you mark done."

Canonical authority. Helix Universal Constitution §11.4.97. Herald §108.l is the project-binding restatement.

Operative rule (Herald-binding). (A) **Maximum-use-of-idle-time** — every minute of conductor idle time during which progressable, non-externally-blocked work exists is a violation; the conductor dispatches work continuously through the whole idle window, not just at scheduled

wakes (the temporal extension of §108.i / §11.4.94 and §108.k / §11.4.96). (B) **Progress-update cadence** — emit concise (1–3 line) operator-facing progress updates at milestone boundaries with NO prompt required: every HEAD advance (what landed), every subagent return (integrated), every constitutional anchor propagated, every captured-evidence artefact (`docs/qa/` / `qa-results/` path), every Issues→Fixed / Obsolete closure. (C) **Continuous physical-proof gathering** per §11.4.5 / §11.4.6 / §11.4.69 (and §107 / §107.x) — every closed item carries positive captured evidence committed alongside the closure; a "done" with no artefact is a §11.4 / §107 PASS-bluff. (E) Idle is permitted ONLY when every item is genuinely blocked on an external dependency, the operator issued `STOP`, or §12 host-session-safety demands it. No `--idle-permitted` / `--silent-progress` / `--evidence-later` escape exists.

Propagation (§108.j–§108.l). These three restatements are summarised — citing this section as the canonical Herald source — in Herald's `CLAUDE.md` and `AGENTS.md` (and `QWEN.md` when present). The universal mandates at Helix §11.4.95 / §11.4.96 / §11.4.97 are cascaded into the Helix-stack submodules per §1.1 multi-file propagation discipline. The literal anchors `11.4.95` / `11.4.96` / `11.4.97` are present in all three Herald root docs.

Non-compliance is a release blocker (§108.j–§108.l). Each sub-clause carries its inherited no-escape-hatch posture: no `--gitignore-db` / `--regenerate-on-clone` (§108.j), no `--idle-during-build` / `--skip-parallel-catalogue` (§108.k), no `--idle-permitted` / `--silent-progress` / `--evidence-later` (§108.l).

§108.m. Full-Automation Anti-Bluff Mandate — Live tests MUST be re-runnable end-to-end without manual intervention (extends Universal §11.4.98; Helix 2026-05-28)

Forensic anchor — verbatim user mandate (2026-05-28):

> "Make sure we have full automation testing of all scenarios with real bot, main group and users without any manual intervention or contribution of real user! Everything MUST BE fully automatic and autonomous! These tests MUST BE able to rerun endless times when needed! This is important to be done like this! It is critical! Continue all work and make this happen! We need such full automation testing so the whole System MUST BE fully valoidated and verified before it is integrated to our main projects! Make sure there is no false positives in testing! Every test and its results MUST obtain real proofs of everything working! No bluff is allowed!"

Canonical authority. Helix Universal Constitution §11.4.98. Herald §108.m is the project-binding restatement.

Operative rule (Herald-binding). Every Herald test — unit / integration / e2e / Challenge / stress / chaos / live — MUST be fully self-driving end-to-end with NO human action during execution (operator typing a Telegram message, hand-triggering a webhook, clicking a UI, attaching a file, anything beyond test startup → PASS / FAIL report). A test requiring manual action during execution is **by definition a §11.4 / §107 PASS-bluff at the automation layer**, regardless of how thorough the manual run is — it cannot run in CI, cannot validate regressions between manual runs, and the human dependency masks drift. (A) **Single permissible exception** — one-time credential bootstrap OUTSIDE test execution (`.env` populated from a vault, shell exports in `~/.bashrc`, OAuth approval at first install, MTProto session activation at first run) — configuration, not test driving. (B) **No "operator MUST type a message" prompts** in `tests/test_*.sh` or `_integration_test.go` — Herald drives programmatically (MTProto user-account for Telegram, real-user-API for Slack, IMAP-test-account for email, webhook fixture, in-process loopback; never human keystrokes during test execution). (C) **No `claude --resume <UUID>` collisions** — test runs MUST use a dedicated test-only session UUID that the production / dev session is NOT simultaneously using. Herald 2026-05-28 lesson

encoded: same-UUID resume returns silent exit -1 with no output. (D) **No 60-second human-response windows** — these are §11.4.50 determinism violations: a single test invocation's PASS / FAIL depends on a human reaction-time variable, not on the code under test. (E) **Re-runnability proof** — every live test MUST PASS at `-count=3` consecutive automated invocations with self-cleaning state (persistent side effects — chats, files, DB rows, queued events — cleaned by the test in `defer`/`teardown`). (F) **§108.m obsolescence audit** — every existing Herald test classified COMPLIANT vs NON-COMPLIANT (release-gate item); currently NON-COMPLIANT and scheduled for MTProto-driven rewrite under Wave 8 Track B: `TestSubscribe_LiveBotAPI`, `tests/test_wave6_live_loop.sh`, Wave 6.5 lifecycle scenarios. (G) **No false-positive PASS** — silent-skip-as-PASS forbidden, stale-evidence forbidden, §11.4.3 SKIP-with-reason is correct. Composes with §107 (anti-bluff) + §107.x (docs/qa evidence mandate) + §108.a (stress + chaos) + §108.b (endless-loop autonomous) + §108.d (background-test) + §108.i (zero-idle parallel). No `--manual-test-OK` / `--skip-114-98-audit` / `--bluff-tolerance-temporary` escape exists.

Propagation. §108.m is restated — citing this section as the canonical Herald source — in Herald's `CLAUDE.md`, `AGENTS.md`, `QWEN.md`. The universal mandate at Helix §11.4.98 is cascaded into the Helix-stack submodules per §1.1 multi-file propagation discipline. The literal anchor `11.4.98` MUST appear in all three Herald root docs; the §11.4.98 pre-build gate `CM-COVENANT-114-98-PROPAGATION` (when implemented) enforces this.

Non-compliance is a release blocker. A commit that adds or modifies a test that requires manual human action during execution is blocked at release-gate. A NON-COMPLIANT test that has not been rewritten within 30 days of classification graduates to §108.e (§11.4.90 Obsolete) and is removed from the active test suite (not deleted — preserved with `Obsolete-Details:` citing §108.m / §11.4.98 as the obsolescence reason).

§108.n. Latest-Source Documentation Cross-Reference Mandate (extends Universal §11.4.99; Helix 2026-05-28)

Forensic anchor — verbatim user mandate (2026-05-28):

> "Make sure we ALWAYS check against latest versions of services we use web / online docs before creating instructions! This situation is illustration of how we can misguide ourselves or get banned! Add this all important generic / general points as proper mandatory rules when we are creating documentation or guides! These are mandatory rules / constraints and the result is consistency and safety of created instructions, guides and manuals!"

Canonical authority. Helix Universal Constitution §11.4.99. Herald §108.n is the project-binding restatement.

Anchoring case study (Herald 2026-05-28). The first-draft MTProto setup guide (commits `35fc10c` / `fb00354` / `f089dd6`) recommended VoIP / Google Voice / Twilio / TextNow numbers as a budget-friendly fallback AND omitted the `recover@telegram.org` pre-login email step. Both directly contradicted (a) Telegram's official docs at `https://core.telegram.org/api/obtaining_api_id` and (b) the gotd/td maintainer's "How to not get banned?" guidance vendored at `submodules/gotd-td/.github/SUPPORT.md`. An operator following the original guide could have had their Telegram account permanently banned. The corrected guide landed at commit `8470ba7` after a forced cross-reference. This is forensic evidence that misguidance-by-stale-docs is the same severity class as a §11.4 / §107 PASS-bluff at the documentation layer.

Operative rule (Herald-binding). Every Herald operator-facing instruction document — `docs/requirements/blockers/*.md`, `docs/guides/*.md`, README operator-action sections, troubleshooting cookbooks, `OPERATOR_CREDENTIALS.md`, `MESSENGER_CHANNELS.md`, `TELEGRAM.md`, the present `HERALD_CONSTITUTION.md`, integration setup walkthroughs, credential acquisition guides, security configuration docs — MUST be cross-referenced against the LATEST official online documentation of every external service / library it touches BEFORE the commit lands.

(A) Pre-commit cross-reference workflow. The Herald author MUST:

1. **Fetch the latest official online docs** of every third-party service / library the new document references via WebFetch / MCP / direct browsing / equivalent authoritative real-time source. Do NOT rely on training data, memory, prior assumptions, or older committed Herald docs as the source of truth.
2. **Cross-reference each instruction in the new document** against that source. For each step the operator will take, verify: (a) the service still supports that action; (b) form fields / parameters / endpoints are still the same; (c) any new constraints, deprecations, ToS changes, or warnings the service published since the doc's last verification.
3. **Seek secondary authoritative sources** when the official documentation is sparse / silent on a critical requirement. Examples: the library maintainer's `submodules/<lib>/{README,SUPPORT,SECURITY}.md`, the service's official changelog / blog, the service's official support channels' responses, community-vetted FAQs.
4. **Cite source URLs + date checked** at the bottom of the document in a `## Sources verified` section. Example: `Sources verified 2026-05-28: https://core.telegram.org/api/obtaining_api_id + submodules/gotd-td/.github/SUPPORT.md`.
5. **Cite the cross-reference in the commit message footer** as a `Sources verified <date>: <urls>` line so the audit trail is reachable via `git log`.

(B) Negative-finding documentation is required. If the cross-reference reveals the official source is silent / contradictory / outdated, the Herald document MUST explicitly note that gap so the next reader does not assume the absence of contradiction means authoritative agreement. The author MAY proceed with the best-available secondary sources, but MUST document the methodology.

(C) Re-verification cadence.

1. **6-month default staleness.** Herald documents older than 6 months without re-verification are STALE — operators MUST NOT trust them for fresh action without re-running the cross-reference.
2. **90-day staleness for risk-classified services** (per §(D) below).
3. **Triggered re-verification** before being cited as the authority for an operator-action campaign, at every vN.0.0 Herald release boundary, on service breaking-change announcements, and when an operator reports an error following the guide (auto-triggers re-verification).

(D) Service-specific risk-classifications for Herald. Herald documentation for the following service families MUST include explicit safety warnings cross-referenced against the latest published policies, with a `Sources verified` date never older than 90 days:

Service family Herald uses	Risks Herald documentation MUST address
Telegram (Bot API + MTProto) — `commons_messaging/channels/tgram`, `qaherald/internal/`	Anti-abuse-system observation; one-phone-one-app-id limits; ban-on-VoIP policies; rate-limit

Service family Herald uses	Risks Herald documentation MUST address
`mtproto`, `submodules/telebot`, `submodules/gotd-td`	floods; user-impersonation risks; pre-login `recover@telegram.org` declaration emails.
Slack — `commons_messaging/channels/slack`, `submodules/slack-go`	Token revocation policies (xoxb-, xapp- prefixes); rate-limit Tier 4 endpoints; Socket Mode app-token requirements; OAuth scopes; workspace owner permissions.
Claude Code / Anthropic API — `commons_messaging/dispatch/claude_code`	ToS violation triggers; rate-limit + quota policies; session UUID handling (Herald 2026-05-28 lesson: dev-session collision); data-retention defaults; PII-in-prompt risks; model-name pinning (`claude-opus-4-7`).
Postgres / Redis / container infrastructure — `commons_infra`, `commons_storage`, `submodules/database`, `containers/`	Volume-data-loss policies on container destroy; password-rotation flows; SQLSTATE error semantics; migration ordering.
Code-hosting (GitHub / GitLab / GitFlic / GitVerse) — `upstreams/` mirror scripts	Token-leak revocation policies; force-push to protected-branch policies; rate-limit-on-API-tokens; secret-scanning bot behaviour.
OS / package managers — `apt`, `brew`, `pip`, `pandoc`, `weasyprint`, `go modules`	Supply-chain compromise vectors; signature verification; lockfile discipline; mirror-trust policies.

This list is **NOT exhaustive** — when a new external service is documented in Herald, the §108.n author judges whether the service has comparable risk surface; if yes, the same safety-warning requirement applies.

(E) Composition. §108.n composes with §107 (anti-bluff), §107.x (docs/qa evidence), §107.y (working-tree quiescence), §108.a (stress + chaos), §108.b (endless-loop autonomous), §108.d (background-test), §108.i (zero-idle parallel), §108.l (max-idle + progress-cadence), §108.m (full-automation testing), and Universal §11.4.92 Pass 4 (deep-research). The §108.n cross-reference is INDEPENDENT of §11.4.92 Pass 4 deep-research — the agent CANNOT cite §11.4.92 as a substitute for §108.n; the two pass independently.

Propagation. §108.n is restated — citing this section as the canonical Herald source — in Herald's `CLAUDE.md`, `AGENTS.md`, `QWEN.md`. The universal mandate at Helix §11.4.99 is cascaded into the Helix-stack submodules per §1.1 multi-file propagation discipline. The literal anchor `11.4.99` MUST appear in all three Herald root docs; the §11.4.99 pre-build gate `CM-COVENANT-114-99-PROPAGATION` (when implemented) enforces this.

Non-compliance is a release blocker. A commit that adds or modifies operator-facing instruction documentation without (a) a `## Sources verified <date>: ...` footer in the document itself AND (b) a `Sources verified <date>: ...` line in the commit-message footer is blocked at release-gate. A Herald document with operator-actionable steps that becomes stale (>6 months default; >90 days for risk-classified services per §(D)) graduates to §108.e (Universal §11.4.90 Obsolete) after the 30-day grace window with `Obsolete-Details: Reason=stale-documentation; Superseding-item=<replacement-doc-or-rewrite>`. No `--skip-source-check` / `--docs-freshness-optional` / `--cite-sources-later` / `--trust-prior-doc-as-authoritative` escape exists.

§108.o. Video color + visual-quality fidelity mandate (extends Universal §11.4.100; Helix 2026-05-28)

Forensic anchor — verbatim user mandate (2026-05-28):

> "We MUST check for all content being played — all videos — video streams from the internet, or offline videos we play, from any of applications — video streaming applications or video players, that all colors are fine and proper! We MUST HAVE same approach to colors like to our sound quality! EVERYTHING MUST be cutting edge, highest quality! ... No false positives or false negatives or bluff(s) of any kind!"

Canonical authority. Helix Universal Constitution §11.4.100. Herald §108.o is the project-binding restatement.

Herald applicability — non-applicable-but-cite. Herald is a messaging / notification system with **NO video-playback surface**:

- `pherald listen` (`pherald/internal/inbound/`) downloads video attachments via the HRD-005 attachment pipeline as opaque sha256-content-addressed blobs to `~/.herald/inbox/<sha>.<ext>`. The handler validates `sha256 + content-length + MIME` and references the file path to the Claude Code dispatcher — it never decodes, never renders, never displays a frame.
- The Claude Code dispatcher (`commons_messaging/dispatch/claude_code/`) receives the file path as text inside the `<<<HERALD-DISPATCH-v1>>>` envelope; downstream rendering (if any) is Claude Code's responsibility, not Herald's.
- The `qaherald` MTProto driver (Wave 8 Track B) likewise treats video attachments as bytes-with-hash for round-trip integrity — never as a rendered frame.

The mandate therefore binds **latently**: if Herald ever ships a video-rendering surface (preview thumbnails in a future dashboard flavor, an in-app player, an inline diff viewer for video-attachment review), §108.o + §11.4.100 apply in full at that point — every video-playback PASS will then need a captured-frame deep-analysis artefact proving ΔE2000 + RGB/HSV histogram correlation + no pale/over-saturation/hue-shift + gamma/luma fidelity (BT.601/709/2020 + full-range-vs-limited-range explicit) + sharpness (Laplacian-variance) + aspect-ratio + FPS+speed + continuity (no freeze ≥1s SSIM>0.99). Metadata-only PASS forbidden; comparison harness MUST itself be golden-pair + bad-pair self-validated (no false-positive AND no false-negative). The §108.o helper-script contract (when Herald ever needs one) lives under `scripts/video_fidelity.sh` per the §11.4.100 helper-contract template (`vf_extract_source_frame`, `vf_capture_rendered_frame`, `vf_assert_delta_e2000`, `vf_assert_histogram_correlation`, `vf_assert_no_pale_no_oversaturation`, `vf_assert_sharpness`, `vf_assert_aspect_ratio`, `vf_assert_fps_speed`, `vf_assert_continuity_no_freeze`, `vf_obstruction_ocr_census`).

Cascade-parallel. §108.k (Universal §11.4.96 "Herald has no AOSP build, but the principle binds") is the structural twin — same restatement-with-non-applicability-but-cite shape: the cascaded mandate does not have a direct Herald-side implementation today, but the literal anchor is propagated so the constitutional inheritance contract is satisfied AND the principle's surface — the no-bluff bar for any future feature in the same problem domain — is established up-front.

Propagation. §108.o is restated — citing this section as the canonical Herald source — in Herald's `CLAUDE.md`, `AGENTS.md`, `QWEN.md`. The universal mandate at Helix §11.4.100 is inherited via the §11.4.35 parent-discovery contract; the literal anchor `11.4.100` MUST appear in all three Herald root docs. The upcoming Helix-side pre-build gate `CM-`

`COVENANT-114-100-PROPAGATION` enforces this literal-anchor presence across the canonical consumer fleet (paired §1.1 mutation: strip the anchor → gate FAILs).

Composition. §108.o composes with §107 (anti-bluff), §108.a (Universal §11.4.85 stress + chaos — fidelity holds under sustained playback + fault injection, not just on the first frame, IF Herald ever has a playback surface), §108.m (Universal §11.4.98 full-automation testing — fidelity analysis runs without operator presence), §108.n (Universal §11.4.99 latest-source documentation — the tolerance thresholds and capture mechanisms must be cross-referenced against the latest official codec / display / OS documentation before publication).

Non-compliance is a release blocker. No `--skip-color-fidelity`, `--no-frame-analysis`, `--metadata-video-pass-suffices`, `--color-check-later`, `--source-comparison-optional` flag exists. This applies prospectively the moment Herald ships any video-rendering surface; until then, the non-applicability is documented HERE (this very §108.o restatement) — the absence-of-implementation is itself the §108.o evidence shape.

§108.p. Docs Chain documentation-sync mandate (extends Universal §11.4.106; Helix 2026-05-29)

Short tag: `docs-chain documentation sync.`

Herald's derived documentation artefacts — every `.md`'s `.html`/`.pdf`/`.docx` siblings, the `*_Summary.md` parity docs (§11.4.12/§11.4.53/§11.4.56), the README doc-link section (§11.4.57), the workable-items DB⍰MD projection (§11.4.93/§11.4.95), and the CONTINUATION exports (§12.10) — MUST be kept in sync by the shared **Docs Chain** engine (`digital.vasic.docs_chain`), a content-hashed incremental DAG sync engine consumed BY REFERENCE through the constitution submodule (Phase-6 distribution operator-gated per §11.4.66), NOT by ad-hoc per-doc scripting. The engine **refuses to fake success** (a missing transform tool → honest rollback, exit 3, no fabricated output) and **refuses silent merges** (both endpoints of a `sync` edge dirty → conflict, exit 2, zero writes) — making §11.4.6 (no-guessing) and §11.4.50 (deterministic consistency) mechanical rather than aspirational, and emitting per-run evidence to `qa-results/docs_chain/<run-id>/` (§11.4.69).

Herald-applicability classification: APPLICABLE (integration in progress). Unlike §108.o (video, non-applicable-but-cite), Docs Chain directly governs Herald's documentation surface. Herald's ~76 tracked markdown docs each carry html/pdf/docx siblings produced today by the regen-all `scripts/export_docs.sh`. The migration plan docs/research/docs_chain/HERALD_DOCS_CHAIN_PLAN.md wires them through docs_chain `exec:` transforms that preserve Herald's exact logo-CSS pandoc flags (`-f gfm -t html5 --css print.css`; weasyprint pdf; pandoc docx) for byte-compatible, zero-regen-drift output across ~223 siblings, gaining content-hash incremental recompute (only-changed) + a `verify` drift-gate (planned `scripts/e2e_bluff_hunt.sh` invariant E146).

Open integration prerequisites (honest — not yet closed). (1) The Phase-4 docs_chain CLI (`sync`/`verify`/`doctor`/`graph`) landed 2026-05-31. (2) Gap G6: `exec:` transforms run against staged temp files, but Herald's html export uses relative logo-CSS / `<img>` paths — the exec-adapter path/cwd contract MUST be read from final source and the wrappers MUST reconstruct absolute asset paths before authoring (§11.4.6 no-guessing). (3) A binary-hash `verify` defect found by dogfooding — binary node-kinds (docx/pdf) hashed inconsistently between the sync-record and verify-check paths, producing false-positive STALE — is being fixed in docs_chain with a pinned regression test; until it lands the drift-gate is trusted for html only.

Canonical authority. Helix Universal Constitution §11.4.106. Herald §108.p is the project-binding restatement.

Propagation. §108.p is restated — citing this section as the canonical Herald source — in Herald's `CLAUDE.md`, `AGENTS.md`, `QWEN.md`. The universal mandate at Helix §11.4.106 is inherited via the §11.4.35 parent-discovery contract; the literal anchor `11.4.106` MUST appear in all three Herald root docs. The Helix-side pre-build gate `CM-COVENANT-114-106-PROPAGATION` enforces this literal-anchor presence (paired §1.1 mutation: strip the anchor → gate FAILs).

Composition. §108.p composes with §107 (anti-bluff — a `verify` PASS is real evidence the siblings match their sources, never a metadata claim), §108.m (Universal §11.4.98 full-automation — `sync`/`verify` run with no operator presence), §108.n (Universal §11.4.99 latest-source documentation — the engine keeps exports in sync but the operator still authors the source + revision header), and §108.j/§108.k (Universal §11.4.95/§11.4.96 — the workable-items DB⍰MD sync edge is the same engine, pending DB materialization, gap G2).

Non-compliance is a release blocker. No `--skip-docs-sync`, `--exports-stale-OK`, `--regen-later` flag exists; once the drift-gate is live, a doc whose committed siblings are stale vs their source is a §107 documentation-layer bluff.

---

§109. Participant identity, attribution & notification-tagging (operator mandate, 2026-05-31; inherited from HelixConstitution per §11.4.35)

Forensic anchor — verbatim operator mandate (2026-05-31):

> every messenger must relate messages to **participants (Subscribers/Users)**; workable items gain `created_by` + `assigned_to`; notifications **@-tag** the right participant per a fixed rule matrix; the same logical person may have a **different username on every messenger**.

Canonical authority. The single authoritative contract every implementation stream codes against is [`docs/design/PARTICIPANT_ATTRIBUTION.md`](docs/design/PARTICIPANT_ATTRIBUTION.md). The mandatory rules are restated (root definitions) in HelixConstitution `Constitution.md` / `CLAUDE.md` / `AGENTS.md` / `QWEN.md` and inherited per §11.4.35; Herald §109 is the project-binding restatement. Herald **restates + cites**, it does NOT redefine or weaken.

§109.1. Identity model — logical participant + per-channel handle

A **Participant** (logical Subscriber/User) is one person/agent, with a potentially DIFFERENT username on every messenger. Backed by the existing PG tables:

- `subscribers` — the logical party: `handle` (canonical, messenger-neutral), `display_name`, `kind ∈ {human, agent, service}`.
- `subscriber_aliases` — the per-channel handle: `subscriber_id`, `channel`, `channel_user_id`, + **NEW** `username TEXT` (the per-channel `@handle` used for tagging — distinct from `channel_user_id`, which is the chat/user id). `UNIQUE (channel, channel_user_id)`.

The **canonical handle** = the string stored in items' `created_by` / `assigned_to`. Closed set:

- `Claude` — the system agent (reserved sentinel; `kind=agent`). NEVER tagged.

- a human's **canonical handle** — defaults to their Telegram `@username` (Telegram is the primary messenger) but is messenger-neutral; per-channel `@username`s are resolved via `subscriber_aliases`.

§109.2. Operator env var

The **operator** is the one human who drives the system via the Claude Code CLI. Designated by env var, NOT a DB flag:

Env var	Example value	Meaning
`HERALD_TGRAM_OPERATOR_USERNAME`	`@milos85vasic`	The operator's Telegram `@userna` (primary messenger
`HERALD_<CHANNEL>_OPERATOR_USERNAME`	`HERALD_SLACK_OPERATOR_USERNAME=…`	Per-messenger generaliza for any oth channel.

The operator's **canonical handle** = their Telegram operator username (e.g. `@milos85vasic`). The operator is a normal Participant whose handle equals the operator env value. `IdentityResolver.OperatorHandle()` returns this.

§109.3. Attribution rules — who sets `created_by` / `assigned_to`

`created_by` (who opened/assigned the item):

- Opened via the **Claude Code CLI prompt** (operator-driven) → `created_by = OperatorHandle()`.
- Opened by **System/Claude** detecting an issue/task/improvement/missing-feature → `created_by = "Claude"`.
- Received **through Herald** (a subscriber message) → `created_by =` the sender's resolved canonical handle (via `ResolveSender` from the message's `@username` + other data).

`assigned_to`:

- **Default** = `OperatorHandle()` (the operator's canonical handle).
- May be overridden explicitly (e.g. a prompt or message that assigns to `@someoneelse`).

Both columns store the **canonical handle string** (self-contained in the SSoT/MD).

§109.4. Notification-tagging matrix — who gets @-mentioned

On any workable-item event, the outbound notification dispatched to each messenger channel/ group @-tags the participant(s) who must be aware, resolved to that channel's `@username`:

Condition	@-tag?	Rationale
`assigned_to` is a human handle AND `assigned_to != Operator`	**yes** — tag `assigned_to`	the assignee must be made aware
`created_by` is a human handle AND `created_by != Operator` AND `created_by != "Claude"`	**yes** — tag `created_by`	a non-operator subscriber opened it
handle == `"Claude"`	**no**	`Claude` is the system; never tagged
handle == Operator	**no**	the operator drives the system; no self-ping

De-dup the resulting set; for each mention resolve `UsernameFor(handle, channel)` and **skip** if the participant has no alias on that channel (you cannot tag someone not on that messenger). This satisfies the operator's stated cases: assigned-to-Operator → no tag; opened-by-Operator-assigned-to-another → tag the assignee; opened-by-a-non-Operator-non-Claude subscriber → tag them.

§109.5. Storage, wiring & anti-bluff

- **Storage.** The workable-items SSoT `items` table gains `created_by TEXT NOT NULL DEFAULT ''` + `assigned_to TEXT NOT NULL DEFAULT ''` (parser reads `**Created-By:**` / `**Assigned-To:**`; renderer writes them; byte-identical round-trip; `validate` accepts empty for legacy). `commons_workable.Item` gains `CreatedBy` + `AssignedTo`; the change-feed emits `item.field.changed` for those fields. Markdown trackers add **Created-By** + **Assigned-To** columns/fields. PG migration adds `subscriber_aliases.username`.
- **Wiring.** Inbound (`pherald/internal/inbound`) sets `created_by` via `ResolveSender`, defaults `assigned_to = OperatorHandle()`; Claude-opened items use `created_by = "Claude"`. Outbound (`pherald/internal/workflow`) calls `MentionsFor` and prepends/appends the resolved `@usernames` to the dispatched body per channel; the `tgram` adapter renders a mention as `@username` (other adapters render their channel's mention syntax — future).
- **Anti-bluff (§107 / Helix §11.4).** Every layer ships unit + integration + E2E + full-automation tests with real captured evidence under `docs/qa/<run-id>/`: real SQLite round-trip with the new columns (byte-identical); a real `IdentityResolver` over real `subscribers`/`subscriber_aliases`; the tagging matrix proven by a truth-table test covering every cell + a per-cell-flip mutation that must FAIL; an E2E with a real item event → a real dispatched message whose body contains exactly the expected `@usernames`, plus a NEGATIVE case proving the Operator is NOT tagged.

Non-compliance is a release blocker. Documenting behaviour without the captured evidence above is itself a §107 PASS-bluff. No `--skip-attribution`, `--tag-later`, `--operator-tag-OK` flag exists.

---

§110. Intent recognition & clarification (operator mandate, 2026-05-31; inherited from HelixConstitution §11.4.105 per §11.4.35)

Forensic anchor — verbatim operator mandate (2026-05-31):

> users must NOT need to know command syntax (no `COMMAND: …`). They send a clear natural-language message; the System determines the intent. The System

recognizes the commands it has; if none matches it infers the exact intent; if it is *totally unable* it replies, tags the user (`@user …`), and asks to clarify precisely. We MUST always do our best to determine exact intent so we never annoy end users. This is a CORE part of the System.

Canonical authority. The single authoritative contract every implementation stream codes against is [`docs/design/INTENT_RECOGNITION.md`](). The mandatory rules are restated (root definitions) in HelixConstitution `Constitution.md` / `CLAUDE.md` / `AGENTS.md` / `QWEN.md` as §11.4.105 (the root-§ being added on the constitution stream) and inherited per §11.4.35; Herald §110 is the project-binding restatement. Herald **restates + cites**, it does NOT redefine or weaken.

§110.1. No command syntax required

Subscribers speak **plain natural language**. There is no `COMMAND:` prefix, no fixed grammar, and nothing the end user must memorize. Herald's job is to determine the intent from the message itself. A subscriber writing "can you close ATM-123 please" is exactly as valid as any other phrasing — the System maps it to the right action.

§110.2. Three-tier intent resolution (the mandatory discipline)

Every inbound subscriber message is resolved to exactly one action via three tiers, in order — the **first tier that succeeds wins**:

Tier	Mechanism	Behaviour
Tier 1 — command recognition	A deterministic `CommandRecognizer` in `pherald/internal/inbound`, tried BEFORE any LLM dispatch.	Maps a clear natural-language imperative to a structured action WITHOUT an LLM round-trip (fast-path; no prefix). Deliberately CONSERVATIVE — only a confident match (clear imperative verb + resolvable target) fast-paths; otherwise it returns "no match" and defers to Tier 2. A false command-match is worse than deferring to the LLM.
Tier 2 — intent inference	The Claude Code dispatch (the LLM) when no command matched.	Infers the intent from the message and returns a `<<<HERALD-REPLY>>>` action. The `<<<HERALD-DISPATCH-v1>>>` envelope INSTRUCTS the LLM to recognize Herald's command set, map natural language to the right action, and NEVER guess.
Tier 3 — clarify (fallback)	`action="clarify"` when neither a command nor a confident intent can be determined.	The System REPLIES to the original message, TAGS the sender (`@username`, resolved via the §11.4.104 `IdentityResolver.UsernameFor` — fall back to the raw sender handle if no alias), and asks a precise clarifying question naming the candidate intents. No guessing, no silent drop.

Tier 3 is the **anti-annoyance guarantee**: the subscriber is never ignored and never has to learn syntax — at worst they get a friendly, specific "@user, did you mean X or Y?" rather than a generic "I didn't understand".

§110.3. The command set Tier 1 recognizes (natural-language → action)

The recognizer maps unambiguous imperatives (with an `ATM-NNN`/item id where relevant) to the EXISTING inbound actions — no special prefix, case-insensitive, phrasing-tolerant:

Natural-language intent (examples)	action	fields
"close ATM-123", "mark ATM-5 fixed/done/resolved"	`item.update`	status=closed/fixed
"set ATM-9 to in progress", "ATM-9 is blocked"	`item.update`	status=<parsed>
"assign ATM-5 to @bob", "give ATM-5 to @bob"	`item.update`	assigned_to=@bob
"open a bug: <title>", "create a task: <title>", "new feature request: <title>"	`issue.open`	type+title (created_by=sender)
"investigate ATM-7", "look into ATM-7"	`investigation.start`	atm_id
"status of ATM-9?", "what's ATM-9?"	`reply` (query)	atm_id
anything conversational / a question	`reply`	—
ambiguous / unparseable intent	`clarify`	question

§110.4. The `clarify` action — never guess, never ignore

`<<<HERALD-REPLY>>>` gains `action: "clarify"` carrying a precise `question`. On `action=clarify`, the inbound handler sends a reply to the original message (quoting/threading it) whose body is `@<sender-username> <question>` — the sender resolved to their per-channel `@username` via the §11.4.104 `IdentityResolver`. The question MUST be specific (name the candidate intents), never a generic "I didn't understand".

Two hard rules bind every tier:

- **Never guess an action.** A wrong action is worse than a clarifying question (§11.4.6 no-guessing). The LLM (Tier 2) is instructed — via the envelope/system prompt — to RETURN `action=clarify` with a precise question whenever it cannot determine the intent with confidence, rather than guess.
- **Never ignore a message.** Every inbound message resolves to exactly one action; genuine ambiguity reaches Tier 3 and is always answered, tagged, and threaded back to the sender.

§110.5. Wiring & anti-bluff

- **Wiring.** `pherald/internal/inbound` runs the `CommandRecognizer` (Tier 1) before the Claude Code dispatch (Tier 2); a parsed `Reply.Action` of `clarify` (Tier 3) routes to a `clarifyHandler` that tags the sender and asks. `commons_messaging/dispatch/claude_code` carries the additive envelope instruction (§110.2 Tier 2). The clarify reply reuses the §109 / §11.4.104 participant `@username` resolution.
- **Anti-bluff (§107 / Helix §11.4).** Every tier ships unit + integration + E2E + full-automation tests with real captured evidence under `docs/qa/<run-id>/`: a Tier-1 truth-table of natural-language messages → expected action+fields (and the conservative negatives that MUST fall through to "no match"); a Tier-3 E2E where an ambiguous message drives a real dispatch whose recording-sink reply body is EXACTLY `@<sender> <specific question>` (proving the user is tagged + asked, not ignored) plus a NEGATIVE proving a clear command does NOT trigger clarify; a paired

§1.1 mutation gate (break the recognizer's confidence guard so it false-matches, OR drop the clarify tag) that MUST FAIL.

Non-compliance is a release blocker. Documenting intent-resolution behaviour without the captured evidence above is itself a §107 PASS-bluff. No `--skip-intent`, `--guess-OK`, `--clarify-later` flag exists.

Overrides of Universal Constitution

(none — Herald has no exceptions to universal clauses at pre-implementation stage)

Owned-submodule set (per Universal §4)

```
submodules/auth          → git@github.com:vasic-digital/auth.git
submodules/background    → git@github.com:vasic-digital/BackgroundTasks.gi
submodules/cache         → git@github.com:vasic-digital/cache.git
submodules/config        → git@github.com:vasic-digital/config.git
submodules/database      → git@github.com:vasic-digital/database.git
submodules/eventbus      → git@github.com:vasic-digital/EventBus.git
submodules/middleware    → git@github.com:vasic-digital/middleware.git
submodules/observability → git@github.com:vasic-digital/observability.git
submodules/recovery      → git@github.com:vasic-digital/recovery.git
containers               → git@github.com:vasic-digital/containers.git
```

Herald owns 10 vendored submodules — 9 Helix-stack capability modules under `submodules/` (referenced via `replace` directives in consuming Herald modules' `go.mod`, NOT via `go.work`) plus `containers/` (runtime auto-detection + on-demand container boot, consumed directly by Foundation tests + `pherald doctor`). Every one of them carries the §11.4 anti-bluff anchor; per Universal §11.4.74 they are catalogue-aware (`extend|reuse|no-match` discipline). The constitution itself is provided by the parent project, not vendored here — Herald never carries `constitution/` (per §104).

Project-specific remotes

Repo	Remotes
Herald (this repo)	`github`, `gitlab`, `gitflic`, `gitverse` + `origin` (fan-out push to all four)

Notes

Herald's spec is now in V3 (`docs/specs/mvp/specification.V4.md`, ~3900 lines, active) — V1 and V2 are preserved under `docs/specs/mvp/archive/` for historical reference. As project-specific articles mature toward universal status they may move into the Helix Constitution; promotion requires the §11.4 universal-vs-project audit. Default is to keep rules here until the audit clears.